Big Data Analytics: Mini Project 1
Project Title: The Digital Librarian (Distributed Reverse Indexing)

Topic	Weight	Team Size	Duration	Due Date
HDFS & MapReduce	15%	2 Students	12 Days	End of Week 6

---

1. Objective

The primary goal of this project is to provide hands-on experience in applying the Data Science Lifecycle to solve a large-scale text processing problem. Upon completion, students will be able to:

- Architect a distributed storage solution by loading and managing a large corpus of text in the Hadoop Distributed File System (HDFS).
- Design and implement a distributed algorithm using the MapReduce programming model.
- Construct a functional Reverse Index, a core component of modern search engines, for high-speed information retrieval.
- Conduct a quantitative Scalability Analysis by rigorously evaluating the job's performance across varying cluster sizes and analyzing the results using the concepts of Speedup.

2. Problem Statement

In the era of Big Data, traditional methods of data processing become bottlenecks. Sequentially scanning documents (e.g., using the grep command) to find keywords does not scale. As the size of a digital library grows from megabytes to terabytes, the latency for simple keyword searches becomes unacceptable.

You are tasked with building a Reverse Index. This data structure is fundamental to search engines. Instead of mapping a document to the words it contains (forward index), a reverse index maps a word to a list of the documents it appears in, along with its frequency in that document.

Final Output Format:

word1 --> document_a.txt:frequency, document_b.txt:frequency, ...

word2 --> document_b.txt:frequency, document_c.txt:frequency, ...

3. Data Science Lifecycle & Project Requirements

I. Problem Definition & Data Collection

- **Dataset:** Select 10–20 large public-domain books from Project Gutenberg.
- **Storage:** Upload files to HDFS at `/user/student/library/`.
- **Report Requirement:** Justify the use of HDFS block storage and replication for this specific dataset.

II. Data Preparation (Preprocessing)

Before indexing, the data must be cleaned. In your Mapper, implement:

1. **Normalization:** Convert all text to lowercase and remove punctuation.
2. **Stop-Word Removal (Mandatory):** Filter out common words (e.g., *the, is, at, which*).
    - *Implementation:* Use a `stopwords.txt` file and load it into the Mapper's memory.

III. Modeling & Processing (MapReduce Logic)

- **Mapper:** Tokenizes cleaned text and emits (word, document_name).
- **Shuffle & Sort:** The framework groups all document references for a single word.
- **Reducer:** Aggregates counts and constructs the final index entry.
    - *Example Output:* hadoop -> book1.txt:12, book3.txt:5

IV. Evaluation & Performance Analysis (The Experiment)

You must measure the **Horizontal Scalability** of your solution by running the exact same job under three cluster configurations:

1. **1 Node:** Pseudo-distributed mode.
2. **2 Nodes:** Small cluster.
3. **3 to 5 Nodes:** Bigger cluster.

Metrics to calculate:

- **Execution Time (T):** Total seconds from submission to completion.
- **Speedup (S):** S = T1 / Tn (where n is the number of nodes).

---

4. Performance Report Structure

Your final report (PDF) is a crucial component of the submission. It must be well-structured and professionally presented, containing the following sections:

Title Page: Project Title, Student Names, Date, Course Info.

1. Introduction / Problem Definition:

- Restate the problem of sequential scanning and its limitations.
- Explain what a Reverse Index is and who benefits from it (e.g., librarians, researchers, search engine users).

2. Data Science Lifecycle Implementation:

- Storage Analysis: Discuss HDFS block storage, replication, and data locality concerning your chosen dataset.
- Preparation Impact: Analyze how stop-word removal reduced the amount of intermediate data (map output) and how this, in turn, lessened the burden of the "Shuffle" phase (network I/O). Quantify this if possible (e.g., "Stop-word removal filtered out approximately X% of tokens, reducing shuffle size by Y MB").
- Processing Logic: Briefly explain your Mapper, Reducer, and (if implemented) Combiner logic.

3. Scalability Analysis & Results:

- Methodology: Describe your experimental setup (dataset size, cluster configurations, how metrics were recorded).
- Performance Visualization: Include a clear, labeled graph plotting Cluster Size (X-axis) vs. Execution Time (Y-axis) for all three runs.
- Results Discussion:
- Present your calculated Speedup values.
- Analyze the graph: Did you achieve linear speedup? If not, why? (Discuss overheads like network communication, task initialization, and the immutable "sequential portion" of the job as per Amdahl's Law).
- Identify potential bottlenecks observed during the experiment.

4. Conclusion: Summarize your findings on the effectiveness of distributed indexing and the scalability of your solution.

5. Grading Rubric (Total: 15 Marks)

Category	Criteria	Marks
Data Science Lifecycle	**Problem Definition & Prep:** Documenting the "Why" and the impact of Stop-word removal on the lifecycle.	**2**
Functional Correctness	**Reverse Index:** Correct word-to-document mapping and frequency counts.	**5**
Distributed Logic	**HDFS & Framework:** Proper use of HDFS, Block concepts, and Stop-word filtering implementation.	**2**
Performance Engineering	**Testing:** Accurate measurements across 1, 2, and 3+ nodes.	**3**
Analytical Rigor	**Discussion:** Quality of Speedup analysis, overhead identification, and performance graphing.	**2**
Professionalism	**Deliverables:** Code quality, README clarity, and report structure.	**1**

Bonus (+1 Mark): * Implementation of a **Combiner** to optimize local aggregation.

- Analysis of how changing the number of **Reducers** affects shuffle time.

6. Deliverables

Submit a ZIP file containing:

1. **Source Code:** Mapper, Reducer, and Driver code.
2. **Resources:** The stopwords.txt used.
3. **README:** Instructions for compilation and execution.
4. **Performance Report:** The PDF containing the Data Science Lifecycle analysis and scaling graphs.

Due Date:
Thursday, 19 March 2026 (29 Ramadan 1447 AH) – 11:59 PM

Important: Late submissions will not be accepted and will receive zero marks.